ZoSolvers

Zeroth-Order Optimisation Solvers with Random Oracles

User Manual — Version 0.1.0

Amir Ali Farzin

Contents

1 Introduction **3**

2 Installation **4**
2.1 Requirements . . . 4
2.2 Install from Wheel (System-Wide) . . . 4
2.3 Install in a Poetry Environment . . . 4
2.4 Development Install . . . 4
2.5 Running the Demo Scripts . . . 4
2.6 Running the Test Suite . . . 4

3 Package Structure **5**

4 Mathematical Background **6**
4.1 Zeroth-Order Optimisation . . . 6
4.2 Random Oracles . . . 6
4.2.1 Gaussian Oracle . . . 6
4.2.2 Sphere Oracle . . . 7
4.2.3 Efficient Sampling . . . 7
4.3 Minimax Oracles . . . 7

5 Modules **8**
5.1 `utils.py` . . . 8
5.1.1 `is_diagonal` . . . 8
5.2 `minimisation.py` — `ZO_gauss_min` . . . 8
5.2.1 Constructor . . . 8
5.2.2 Methods . . . 9
5.3 `minimax.py` — `ZO_gauss_minmax` . . . 9
5.3.1 Constructor . . . 9
5.3.2 Methods . . . 10

6 Solver Algorithms **12**
6.1 Zeroth-Order Gradient Descent (ZOGD) . . . 12
6.2 Zeroth-Order Extra-Gradient for Minimisation (ZOEGm) . . . 12
6.3 Zeroth-Order Gradient Descent-Ascent (ZOGDA) . . . 12
6.4 Zeroth-Order Extra-Gradient for Minimax (ZOEGmm) . . . 13
6.5 Early Stopping Criteria . . . 13

7 Usage Examples **15**
7.1 Minimisation . . . 15
7.1.1 Basic Unconstrained Minimisation . . . 15
7.1.2 Sphere Oracle . . . 15
7.1.3 Precision Matrix B . . . 15
7.1.4 Box-Constrained Minimisation . . . 15

7.1.5 Growing Mini-Batch (`t = "iteration"`) 16
7.1.6 Early Stopping 16
7.1.7 Extra-Gradient 16
7.2 Minimax 16
7.2.1 Basic Unconstrained Minimax 16
7.2.2 Sphere Oracle with Independent Precision Matrices 16
7.2.3 Projected Minimax 17
7.2.4 Step-Size Ratio τ 17
7.2.5 Extra-Gradient Minimax 17

8 Testing **18**
8.1 Unit and Statistical Tests 18
8.2 Demo Scripts 18

9 Parameter Guide **19**

Chapter 1

Introduction

ZoSolvers is a Python package for zeroth-order (ZO) optimisation. It provides gradient-free solvers for both minimisation and minimax problems using random directional oracles. Two oracle families are supported:

- **Gaussian oracle**: the perturbation direction is sampled from a multivariate normal distribution.
- **Sphere oracle**: the perturbation direction is sampled uniformly from a (generalised) unit sphere.

Both oracle types can be equipped with a *precision matrix* B that shapes the distribution of perturbation directions, enabling preconditioning analogous to quasi-Newton methods.

Chapter 2

Installation

2.1 Requirements

- Python $\geq$ 3.10
- `numpy` $\geq$ 2.2.6
- `matplotlib` $\geq$ 3.7 (for demo scripts only)

2.2 Install from Wheel (System-Wide)

After building the package with `poetry build`, install the generated wheel globally:

```
pip install dist/zosolvers-0.1.0-py3-none-any.whl
```

2.3 Install in a Poetry Environment

```
poetry install
```

2.4 Development Install

```
git clone https://github.com/amirali77farzin/ZoSolvers.git
cd ZoSolvers
poetry install
```

2.5 Running the Demo Scripts

```
poetry run python tests/demo_min.py        # minimisation demo
poetry run python tests/demo_minimax.py    # minimax demo
```

2.6 Running the Test Suite

```
poetry run pytest tests/min_test.py -v
poetry run pytest tests/minimax_test.py -v
```

Chapter 3

Package Structure

```
ZoSolvers/
|-- pyproject.toml          package metadata and dependencies
|-- poetry.lock             pinned dependency versions
|-- .gitignore
|-- README.md
|-- Docs/
|   |--ZoSolvers_Manual.pdf              this document
|
|-- src/
|   `-- ZoSolvers/
|       |-- __init__.py
|       |-- minimisation.py     ZO_gauss_min  (minimisation solvers)
|       |-- minimax.py          ZO_gauss_minmax (minimax solvers)
|       `-- utils.py            shared utility functions
`-- tests/
    |-- conftest.py             pytest path setup
    |-- min_test.py             unit + statistical tests for minimisation
    |-- minimax_test.py         unit + statistical tests for minimax
    |-- demo_min.py             runnable minimisation demo with plots
    `-- demo_minimax.py         runnable minimax demo with plots
```

Chapter 4

Mathematical Background

4.1 Zeroth-Order Optimisation

In zeroth-order optimisation the gradient $\nabla f(x)$ is not available. Only *function evaluations* $f(x)$ can be queried. A *random oracle* approximates the gradient by finite differences along a random direction u:

Forward difference

$$\hat{g}(x) = \frac{f(x + \mu u) - f(x)}{\mu} \cdot c\, u,$$

Backward difference

$$\hat{g}(x) = \frac{f(x) - f(x - \mu u)}{\mu} \cdot c\, u,$$

Central (symmetric) difference

$$\hat{g}(x) = \frac{f(x + \mu u) - f(x - \mu u)}{2\mu} \cdot c\, u,$$

where $\mu > 0$ is the smoothing parameter and c, u depend on the oracle type (Section 4.2).

4.2 Random Oracles

4.2.1 Gaussian Oracle

Sample the direction $u \sim \mathcal{N}(0, B^{-1})$ where $B \succ 0$ is a user-supplied *precision matrix*. The oracle is

$$\hat{g}(x) = \frac{\Delta f}{\mu} \cdot Bu, \qquad \mathbb{E}[\hat{g}(x)] \approx \nabla f_\mu(x),$$

where f_μ is the Gaussian-smoothed objective. Unbiasedness follows from

$$\mathbb{E}\Big[(Bu)(Bu)^\top\Big] = B\,\mathbb{E}[uu^\top]\,B = BB^{-1}B = B, \quad \text{so} \quad B\,\mathbb{E}[uu^\top]\,\nabla f = \nabla f.$$

When $B = I$ the direction is standard Gaussian: $u \sim \mathcal{N}(0, I)$.

4.2.2 Sphere Oracle

Sample s uniformly on $\mathcal{S}^{d-1}$ (unit sphere) via normalising a standard Gaussian draw:

$$s = z/\|z\|, \quad z \sim \mathcal{N}(0, I).$$

Set $u = Ls$ where $L = \mathrm{chol}(B^{-1})$ (Cholesky factor). The oracle is

$$\hat{g}(x) = \frac{\Delta f}{\mu} \cdot d \cdot Bu, \qquad \mathbb{E}[\hat{g}(x)] \approx \nabla f_\mu(x).$$

The factor d (dimension) corrects for $\mathbb{E}[ss^\top] = \frac{1}{d}I$:

$$d \cdot B\,\mathbb{E}[uu^\top]\,\nabla f = d \cdot B \cdot L\tfrac{1}{d}I \cdot L^\top \nabla f = BB^{-1}\nabla f = \nabla f.$$

When $B = I$, $u = s$ is simply a uniform unit-sphere sample.

4.2.3 Efficient Sampling

To avoid recomputing matrix factorisations at every iteration, ZoSolvers caches the sampling structure at construction time:

B	Structure	Cached object
`None`	identity	—
diagonal	`'diagonal'`	$\mathrm{diag}(B)^{-1/2}$
general	`'full'`	$L = \mathrm{chol}(B^{-1})$

4.3 Minimax Oracles

For $\min_x \max_y f(x, y)$, ZoSolvers uses a simultaneous perturbation (SPSA-style) oracle. Directions u_x, u_y are drawn independently; a single finite-difference quotient

$$g = \frac{\Delta f(x + \mu u_x,\, y + \mu u_y)}{\mu \text{ or } 2\mu}$$

is computed. The partial oracles are then

$$\hat{g}_x = c_x \cdot g \cdot B_x u_x, \qquad \hat{g}_y = c_y \cdot g \cdot B_y u_y,$$

with $c_x = d_x$ (sphere) or 1 (Gaussian), and similarly for c_y. Independence of u_x and u_y ensures cross-terms vanish and $\mathbb{E}[\hat{g}_x] \approx \partial_x f_\mu$, $\mathbb{E}[\hat{g}_y] \approx \partial_y f_\mu$.

Chapter 5

Modules

5.1 utils.py

5.1.1 is_diagonal

```python
from ZoSolvers.utils import is_diagonal

is_diagonal(matrix, tol=1e-12)
```

Returns `True` if `matrix` is square and diagonal (up to tolerance `tol`). Raises `RuntimeError` for non-square input. Used internally to select the efficient sampling path.

5.2 minimisation.py — ZO_gauss_min

5.2.1 Constructor

```python
from ZoSolvers.minimisation import ZO_gauss_min

opt = ZO_gauss_min(
    func,                       # callable: f(x) -> float
    x0,                         # array_like: initial point
    h         = 1e-3,           # float: step size
    mu        = 1e-5,           # float: smoothing parameter
    N         = 10000,          # int:   max iterations
    t         = 1,              # int | "iteration": samples per oracle call
    tol_f     = None,           # float | None: relative function tolerance
    tol_g     = None,           # float | None: oracle-norm tolerance
    B         = None,           # ndarray | None: precision matrix
    proj      = None,           # callable | None: projection onto feasible set
    oracle_type = "gaussian" # "gaussian" | "sphere"
)
```

Parameters

Parameter	Description
`func`	Objective function $f : \mathbb{R}^d \to \mathbb{R}$. Must accept a NumPy array.
`x0`	Starting point. Converted to a float array internally.
`h`	Step size. In ZOGD this is the gradient step; in ZOEGm it is the first-stride step.
`mu`	Smoothing parameter $\mu > 0$.
`N`	Maximum number of iterations.
`t`	Number of oracle samples averaged per step. Setting `"iteration"` uses the current iteration number k as the mini-batch size (increasing schedule).
`tol_f`	Stop early when $f(x_k) \leq$ `tol_f` $\cdot f(x_0)$.
`tol_g`	Stop early when the empirical squared oracle norm $\leq$ `tol_g`.
`B`	Precision matrix $B \succ 0$ (inverse of the covariance of the perturbation distribution). `None` $\Rightarrow$ identity.
`proj`	Projection function $\mathrm{proj}_{\mathcal{X}} : \mathbb{R}^d \to \mathcal{X}$. `None` $\Rightarrow$ unconstrained.
`oracle_type`	`"gaussian"`: $u \sim \mathcal{N}(0, B^{-1})$. `"sphere"`: u uniform on B-metric unit sphere.

5.2.2 Methods

`oracle(x, method)`

Returns a single zeroth-order gradient estimate at x using the specified finite-difference `method` (`"forw"`, `"back"`, or `"center"`).

`ZOGD(method="forw")`

Runs Zeroth-Order Gradient Descent. Returns a NumPy array of shape (K, d) where $K \leq N$ is the number of iterates (including x_0).

`ZOEGm(method="forw", gamma=1.0)`

Runs the Zeroth-Order Extra-Gradient method for minimisation. The parameter `gamma` scales the second stride. Returns an array of shape (K, d).

5.3 `minimax.py` — `ZO_gauss_minmax`

5.3.1 Constructor

```python
from ZoSolvers.minimax import ZO_gauss_minmax

opt = ZO_gauss_minmax(
    func,                       # callable: f(x, y) -> float
    x0,                         # array_like: initial minimiser point
```

```
    y0,                          # array_like: initial maximiser point
    h         = 1e-3,            # float: base step size (y-direction)
    tau       = 1,               # float: step-size ratio (x-step = h / tau)
    mu        = 1e-5,            # float: smoothing parameter
    N         = 10000,           # int:   max iterations
    t         = 1,               # int | "iteration": samples per oracle call
    tol       = None,            # float | None: projected-operator-norm tolerance
    B_x       = None,            # ndarray | None: precision matrix for x
    B_y       = None,            # ndarray | None: precision matrix for y
    proj_x    = None,            # callable | None: projection for x
    proj_y    = None,            # callable | None: projection for y
    oracle_type = "gaussian"     # "gaussian" | "sphere"
)
```

Parameters

Parameter	Description
`func`	Saddle-point objective $f : \mathbb{R}^{d_x} \times \mathbb{R}^{d_y} \to \mathbb{R}$, minimised in x, maximised in y.
`x0, y0`	Starting points for minimiser and maximiser.
`h`	Base step size. The y-update uses step h; the x-update uses step h/τ.
`tau`	Step-size ratio $\tau > 0$. Setting $\tau > 1$ makes the x-step smaller than the y-step.
`mu`	Smoothing parameter $\mu > 0$.
`N`	Maximum number of iterations.
`t`	Samples per oracle call, or `"iteration"` for a growing schedule.
`tol`	Stop early when the empirical squared projected-operator norm $\leq$ `tol`.
`B_x, B_y`	Independent precision matrices for x and y directions.
`proj_x, proj_y`	Independent projection functions. `None` $\Rightarrow$ unconstrained.
`oracle_type`	`"gaussian"` or `"sphere"` (applied to both x and y).

5.3.2 Methods

`oracle(x, y, ux, uy, xy, method)`

Returns the partial zeroth-order oracle for variable `xy` (`'x'` or `'y'`) using pre-sampled directions `ux`, `uy`. The SPSA simultaneous perturbation is used so that a single function evaluation pair serves both partial oracles.

`ZOGDA(method="forw")`

Zeroth-Order Gradient Descent-Ascent. Returns `(x_traj, y_traj)`, each a NumPy array of shape (K, d_x) and (K, d_y) respectively.

`ZOEGmm(method="forw", gamma=1.0)`

Zeroth-Order Extra-Gradient for minimax. Two independent oracle samples are drawn per iteration. Returns `(x_traj, y_traj)`.

Chapter 6

Solver Algorithms

6.1 Zeroth-Order Gradient Descent (ZOGD)

Algorithm 1 ZOGD — Zeroth-Order Gradient Descent

Input: f, x_0, step size h, smoothing μ, iterations N, samples t, projection $\mathrm{proj}_{\mathcal{X}}$, oracle type
Output: Trajectory $\{x_k\}_{k=0}^{K}$

```
for k = 0, 1, ..., N − 1 do
    Draw t directions u^(1), ..., u^(t) from the oracle
    ĝ_k ← (1/t) Σ_{i=1}^{t} ĝ(x_k, u^(i), μ)
    x_{k+1} ← proj_X(x_k − h ĝ_k)
    check early-stopping criteria
end for
```

6.2 Zeroth-Order Extra-Gradient for Minimisation (ZOEGm)

The extra-gradient method takes two oracle-based steps per iteration (a *look-ahead* step and an *update* step), which improves convergence on certain problem classes.

Algorithm 2 ZOEGm — Zeroth-Order Extra-Gradient (Minimisation)

Input: f, x_0, h, γ, μ, N, t, $\mathrm{proj}_{\mathcal{X}}$, oracle type
Output: Trajectory $\{x_k\}_{k=0}^{K}$

```
for k = 0, 1, ..., N − 1 do
    // Look-ahead step
    x̂_k ← proj_X(x_k − h ĝ(x_k))
    // Update step
    x_{k+1} ← proj_X(x̂_k − γh ĝ(x̂_k))
    check early-stopping criteria
end for
```

6.3 Zeroth-Order Gradient Descent-Ascent (ZOGDA)

For saddle-point problems $\min_x \max_y f(x, y)$, ZOGDA simultaneously updates both variables using their respective partial oracles.

Algorithm 3 ZOGDA — Zeroth-Order Gradient Descent-Ascent

Input: f, x_0, y_0, h, τ, μ, N, t, $\mathrm{proj}_{\mathcal{X}}$, $\mathrm{proj}_{\mathcal{Y}}$, oracle type
Output: Trajectories $\{x_k\}$, $\{y_k\}$

```
for k = 0, 1, ..., N − 1 do
    Draw directions u_x, u_y independently from the oracle
    ĝ_x ← partial oracle w.r.t. x at (x_k, y_k)
    ĝ_y ← partial oracle w.r.t. y at (x_k, y_k)
    x_{k+1} ← proj_X(x_k − (h/τ) ĝ_x)
    y_{k+1} ← proj_Y(y_k + h ĝ_y)
    check early-stopping criterion
end for
```

6.4 Zeroth-Order Extra-Gradient for Minimax (ZOEGmm)

ZOEGmm extends the extra-gradient idea to the minimax setting. Two independent oracle samples (u_x, u_y) are drawn per iteration: one for the look-ahead step and one for the update step[1].

Algorithm 4 ZOEGmm — Zeroth-Order Extra-Gradient (Minimax)

Input: f, x_0, y_0, h, τ, γ, μ, N, t, $\mathrm{proj}_{\mathcal{X}}$, $\mathrm{proj}_{\mathcal{Y}}$, oracle type
Output: Trajectories $\{x_k\}$, $\{y_k\}$

```
for k = 0, 1, ..., N − 1 do
    Draw (u_x^(1), u_y^(1)) independently
    // Look-ahead step
    x̂_k ← proj_X(x_k − (h/τ) ĝ_x^(1))
    ŷ_k ← proj_Y(y_k + h ĝ_y^(1))
    Draw (u_x^(2), u_y^(2)) independently
    // Update step
    x_{k+1} ← proj_X(x̂_k − (γh/τ) ĝ_x^(2))
    y_{k+1} ← proj_Y(ŷ_k + γh ĝ_y^(2))
    check early-stopping criterion
end for
```

6.5 Early Stopping Criteria

Minimisation (`tol_f`) The algorithm stops when

$$f(x_k) \leq \texttt{tol_f} \cdot f(x_0).$$

Minimisation (`tol_g`) The algorithm stops when

$$\frac{1}{10} \sum_{j=1}^{10} \|\hat{g}(x_k)\|^2 \leq \texttt{tol_g}.$$

For projected problems, the oracle norm is replaced by the squared projected-residual proxy $\|(x_k - \mathrm{proj}_{\mathcal{X}}(x_k - h\,\hat{g}))/h\|^2$.

[1] More details can be found in "Farzin, A.A., Pun, Y.M., Lesage-Landry, A., Diouane, Y. and Shames, I., 2025. Min-max optimisation for nonconvex-nonconcave functions using a random zeroth-order extragradient algorithm. Transactions on Machine Learning Research."

Minimax (`tol`) The algorithm stops when the empirical squared projected-operator norm

$$\frac{1}{10}\sum_{j=1}^{10}\left(\left\|\frac{x_k - \mathrm{proj}_{\mathcal{X}}(x_k - \frac{h}{\tau}\hat{g}_x)}{h/\tau}\right\|^2 + \left\|\frac{\mathrm{proj}_{\mathcal{Y}}(y_k + h\hat{g}_y) - y_k}{h}\right\|^2\right) \leq \texttt{tol}.$$

Chapter 7

Usage Examples

7.1 Minimisation

7.1.1 Basic Unconstrained Minimisation

```python
import numpy as np
from ZoSolvers.minimisation import ZO_gauss_min

def f(x):
    return x[0]**2 + x[1]**2 + x[0]*x[1]

x0  = np.array([5.0, -5.0])
opt = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=10)

# Run zeroth-order gradient descent (centered finite difference)
x_traj = opt.ZOGD(method="center")

print("Final iterate:", x_traj[-1])
print("Final f value:", f(x_traj[-1]))
```

7.1.2 Sphere Oracle

```python
opt_sph = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=10,
                       oracle_type="sphere")
x_traj = opt_sph.ZOGD(method="center")
```

7.1.3 Precision Matrix B

```python
# Diagonal precision matrix (efficient path)
B_diag = np.diag([10.0, 0.01])
opt_diag = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=10, B=B_diag)

# Full precision matrix
B_full = np.array([[20.0, 0.01],
                   [0.01, 0.02]])
opt_full = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=10, B=B_full)
```

7.1.4 Box-Constrained Minimisation

```
proj = lambda x: np.clip(x, -2.0, 2.0)    # project onto [-2, 2]^2

opt_proj = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=10, proj=proj)
x_traj   = opt_proj.ZOGD(method="center")
```

7.1.5 Growing Mini-Batch (t = "iteration")

```
opt = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=500, t="iteration")
x_traj = opt.ZOGD(method="center")
# At iteration k, k oracle samples are averaged.
```

7.1.6 Early Stopping

```
# Stop when f(x) drops to 1% of f(x0)
opt = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=5000, t=10, tol_f=0.01)
x_traj = opt.ZOGD(method="center")
print(f"Stopped after {len(x_traj)} iterations.")
```

7.1.7 Extra-Gradient

```
opt = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=10)
x_traj = opt.ZOEGm(method="center", gamma=1.0)
```

7.2 Minimax

7.2.1 Basic Unconstrained Minimax

```
from ZoSolvers.minimax import ZO_gauss_minmax

def f(x, y):
    return (  x[0]**2 + x[1]**2 + x[0]*x[1]
            - y[0]**2 - y[1]**2 + 1.5*y[0]*y[1] + 3*y[1])

x0 = np.array([10.0, -10.0])
y0 = np.array([ 9.0,  -9.0])

opt = ZO_gauss_minmax(f, x0, y0, h=1e-3, tau=1, mu=1e-8, N=10000, t=5)
x_traj, y_traj = opt.ZOGDA(method="center")

print("Final x:", x_traj[-1])
print("Final y:", y_traj[-1])
```

7.2.2 Sphere Oracle with Independent Precision Matrices

```
B_x = np.diag([2.0, 0.2])
B_y = np.array([[2.0, 0.01],
                [0.01, 0.2]])

opt = ZO_gauss_minmax(f, x0, y0, h=1e-3, tau=1, mu=1e-8, N=10000, t=5,
                      B_x=B_x, B_y=B_y, oracle_type="sphere")
x_traj, y_traj = opt.ZOGDA(method="center")
```

7.2.3 Projected Minimax

```
proj = lambda v: np.clip(v, -7.0, 7.0)

opt = ZO_gauss_minmax(f, x0, y0, h=1e-3, tau=1, mu=1e-8, N=10000, t=5,
                      proj_x=proj, proj_y=proj)
x_traj, y_traj = opt.ZOGDA(method="center")
```

7.2.4 Step-Size Ratio τ

```
# x-step = h / tau = 1e-3 / 2 = 5e-4
# y-step = h       = 1e-3
opt = ZO_gauss_minmax(f, x0, y0, h=1e-3, tau=2, mu=1e-8, N=10000, t=5)
x_traj, y_traj = opt.ZOGDA(method="center")
```

7.2.5 Extra-Gradient Minimax

```
opt = ZO_gauss_minmax(f, x0, y0, h=1e-3, tau=1, mu=1e-8, N=10000, t=5)
x_traj, y_traj = opt.ZOEGmm(method="center", gamma=0.8)
```

Chapter 8

Testing

The `tests/` directory contains two test files and two demo scripts.

8.1 Unit and Statistical Tests

`min_test.py` Tests for `ZO_gauss_min`. Coverage: initialisation attributes and error handling; Cholesky cache correctness; sampling shapes and dispatch; oracle unbiasedness for Gaussian and sphere under identity, diagonal, and full B (statistical tests with 20 000 samples); ZOGD and ZOEGm convergence on a quadratic for all oracle/B combinations; projection feasibility; early stopping; `t="iteration"` mode; small-N safety (no `ZeroDivisionError`).

`minimax_test.py` Tests for `ZO_gauss_minmax`. Same coverage adapted for the minimax setting, plus: independent B_x/B_y caches; partial oracle unbiasedness for both x and y; bilinear cross-term cancellation; `proj_x`/`proj_y` non-mutation test.

Run all tests:

```
poetry run pytest tests/ -v
```

8.2 Demo Scripts

`demo_min.py` Runs eight experiments covering oracle types, finite-difference methods, B matrices, projection, mini-batch schedules, and early stopping. Prints a tabular summary and displays a 2×3 figure of convergence curves.

`demo_minimax.py` Equivalent demonstrations for the minimax solvers, additionally covering the step-size ratio τ and independent B_x, B_y configurations.

Chapter 9

Parameter Guide

`Parameter`	Effect	Typical range
`h`	Controls descent speed. Too large $\Rightarrow$ divergence; too small $\Rightarrow$ slow.	10^{-8} – 10^{-1}
`mu`	Finite-difference step. Smaller reduces bias but increases variance.	10^{-8} – 10^{-3}
`t`	Larger t reduces oracle variance but costs more function evaluations per step.	1 – 50
`B`	Shapes the perturbation distribution. A good B approximates the Hessian.	Problem-dependent
`tau`	(Minimax) Balances x and y convergence rates.	0.5 – 5
`gamma`	(Extra-gradient) Second-stride scale. Smaller $\Rightarrow$ more conservative update.	0.5 – 2
`oracle_type`	`sphere` has lower variance per sample; `gaussian` is simpler and often equally effective.	`"gaussian"`